

Lab 4: Power Analysis and Trend-Detection Review

Instructor notes – Part 1: prep reading, Part 2: podium/circulating cheat sheet

This single file has two parts, both instructor-only (not a student handout). **Part 1** is read beforehand – full worked examples and the reasoning behind the flow. **Part 2** (after the page break, large print) is the copy to have open while circulating during lab.

Where this sits, and why it exists

Lab 4 (“Data generating models”) leans on a wide spread of ideas – some from last week’s distributions material, some genuinely new – compressed into one lab period. Rather than assume it’s all fresh, this is a short (15-20 min) opening pass through six things the lab needs, before students open Act 1. It isn’t a full 50-minute lecture beat; treat it as this lab’s version of `NOTES_Lab1_MathRefresher.Rmd`’s opening refresher, followed by spot-checks with individual groups as they hit each Act.

Not covered again here: *why simulate from a data-generating model in the first place* (validating a fitting procedure, powering a design, building intuition, forecasting) – that’s Monday’s Week 5 Lecture 1 (`NOTES_Week5_Lecture1_SyntheticDataSimulation.Rmd`), delivered the day before. Recap it in one sentence and move on:

“Monday we said one reason to simulate fake data is to check whether a design would even detect a real effect before you go collect it – that’s a power analysis, and today’s review builds the pieces Act 2 needs to run one.”

Deliberately a fresh example throughout: every worked number below uses a different species (greater sage-grouse lek counts, not Lab 4’s toad) and different parameter values than any Act. Same techniques, no spoilers.

Rationale for the flow

Ordered to match how the lab actually uses these ideas: the lognormal distribution first (it’s the noise model behind Act 1 and the optional Act 3), then the regression trick that turns an exponential decline into something `lm()` can fit, then what a confidence interval’s coverage actually means (the thing Act 1’s whole simulation checks), then power analysis in general (the bridge into Act 2), then the two error types and the directional decision rule Act 2’s detection logic depends on, then the independent-trials logic behind Act 2’s detection-probability function, and closing with a coding-practice point (build and test one

function at a time) that isn't a math topic at all but will save a lot of debugging time in Act 2's Part 2.

1. The lognormal distribution (review, ~2 min)

A variable X is **lognormal** if $\log(X)$ is normally distributed: $\log(X) \sim \text{Normal}(m, s)$. Two facts matter more than the formula itself:

- **Right-skewed, mean above median.** The median is e^m ; the mean is $e^{m+s^2/2}$, always a bit larger. A few large draws pull the mean up past the middle.
- **Constant coefficient of variation.** $\text{SD}(X) = \sqrt{e^{s^2} - 1} \times \text{mean}$ – the *spread* is a fixed proportion of the mean, not a fixed number of animals. This is why it's the natural noise model for count indices that can shrink toward zero: the scatter shrinks right along with the signal, rather than swamping a small population the way constant additive noise would.

```
medians <- c(40, 24)           # a lek count index, year 0 and year 20
means <- medians * exp(sigma^2 / 2)
sds <- means * sqrt(exp(sigma^2) - 1)
data.frame(year = c(0, 20), median = medians,
            mean = round(means, 2), sd = round(sds, 2))
```

```
##   year median  mean  sd
## 1    0     40 40.45 6.10
## 2   20     24 24.27 3.66
```

Sticking point to flag: lognormal noise is *multiplicative* ($X = e^m \cdot e^\varepsilon$), not additive – that's exactly why it needs a log transform before it fits into ordinary linear regression, which is the next section.

2. Linearizing an exponential trend (~3 min)

A population or index declining by a constant *proportion* each year follows $I_t = I_0 \lambda^t e^{\varepsilon_t}$, with $\varepsilon_t \sim \text{Normal}(0, \sigma)$. Taking logs:

$$\log(I_t) = \log(I_0) + t \log(\lambda) + \varepsilon_t = \log(I_0) + bt + \varepsilon_t$$

– linear in t , additive noise, constant variance. That's a straight line `lm()` can fit; the raw counts are not.

```
t <- 1:20
set.seed(2201)
idx <- I0 * exp(b * t + rnorm(20, 0, sigma))
m <- lm(log(idx) ~ t)
c(true_b = b, fitted_b = unname(coef(m)["t"]))
```

```
##      true_b      fitted_b
## -0.02554128 -0.02985430
```

Sticking point to flag: the fitted coefficient is the slope of $\log(I)$ versus t , not a percent change directly – converting back takes $\hat{\lambda} = e^{\hat{b}}$ (proportion remaining each year) or $1 - e^{25\hat{b}}$ (percent lost over 25 years, say). Losing track of which scale a number is on is the single most common bug in Act 1.

3. What a confidence interval’s coverage means (~3 min)

A 90% confidence interval is not “a 90% chance the true value is in this one interval” – the true value is a fixed (if unknown) number, not a random one. It means: **if you repeated the whole study many times, about 90% of the intervals you’d compute would contain the true value.** Coverage is a property of the *procedure*, checked by simulating from a *known* truth and seeing how often the interval catches it.

```
check_coverage <- function(R, seed) {
  set.seed(seed)
  hit <- logical(R)
  for (i in seq_len(R)) {
    idx <- I0 * exp(b * t + rnorm(20, 0, sigma))
    m <- lm(log(idx) ~ t)
    ci <- confint(m, "t", level = 0.90)
    hit[i] <- ci[1] <= b && b <= ci[2]
  }
  mean(hit)
}
check_coverage(2000, seed = 5)
```

```
## [1] 0.891
```

Comfortably close to 90%, as it should be when the model is correctly specified. The **Monte Carlo standard error of a proportion**, $\sqrt{p(1-p)/R}$, sets how many replicates (R) you need before trusting a coverage estimate to a given precision – worth a one-line mention since Act 1 asks students to solve for R themselves.

4. Power analysis, in general (~4 min)

Power is the probability that a study design, if run, would correctly detect a real effect – estimated *before* collecting any real data, by simulation. The general recipe, regardless of context:

1. Specify a data-generating model under a **plausible true effect** (not “no effect” – that’s a different check).
2. Simulate one dataset under a **candidate design** (sample size, survey frequency, number of sites...).

3. Apply the actual planned **analysis method** to the simulated data and record whether it correctly detects the effect.
4. Repeat many times; the fraction that detected it is the estimated power for that design.
5. Compare across candidate designs – power almost always trades off against cost, so a power analysis often becomes a small **design optimization** problem, not a single number.

```
power_for_years <- function(T, R, seed) {
  set.seed(seed)
  tt <- 1:T
  detected <- logical(R)
  for (i in seq_len(R)) {
    idx <- I0 * exp(b * tt + rnorm(T, 0, sigma))
    s <- summary(lm(log(idx) ~ tt))$coefficients["tt", ]
    detected[i] <- (s["Pr(>|t|)"] < 0.05) && (s["Estimate"] < 0)
  }
  mean(detected)
}
sapply(c(10, 12, 15, 16, 18), power_for_years, R = 1500, seed = 42)
```

```
## [1] 0.2640000 0.4286667 0.7533333 0.8260000 0.9413333
```

For this (fairly modest) 40%-over-20-years decline, annual monitoring needs on the order of **16 years** before power reaches the common 80% benchmark – a design question a manager would want answered well before committing to a monitoring budget.

Sticking point to flag: power is a property of a *design*, not of data you already have – there’s no “the power of my dataset,” only “the power of a proposed sampling scheme, given an assumed true effect.”

5. Type I/II error and directional decision rules (~2 min)

- **Type I error:** declaring an effect that isn’t really there (false alarm).
 - **Type II error:** failing to declare an effect that really is there (missed detection).
- Power = 1 – Type II error rate.**

When the scientific question has a direction (“was there a *decline*,” not just “was there *any* trend”), the decision rule needs **two** conditions: statistically significant, *and* the estimated direction matches the one being asked about. A naive rule that ignores direction silently inflates false alarms, because half of the *wrong-direction* significant results get counted as hits they shouldn’t be:

```
false_alarm_rate <- function(directional, R, seed) {
  set.seed(seed)
  flag <- logical(R)
```

```

for (i in seq_len(R)) {
  idx <- I0 * exp(0 * t + rnorm(20, 0, sigma))      # true b = 0: no real trend
  s <- summary(lm(log(idx) ~ t))$coefficients["t", ]
  sig <- s["Pr(>|t|)"] < 0.05
  flag[i] <- if (directional) sig && (s["Estimate"] < 0) else sig
}
mean(flag)
}
c(naive = false_alarm_rate(FALSE, 3000, 11),
  directional = false_alarm_rate(TRUE, 3000, 11))

##      naive directional
## 0.04266667 0.02500000

```

Under a **true null** (no real trend), the naive two-sided rule flags a “decline” close to its nominal 5% rate, but the directional rule cuts that roughly in half – exactly what you’d expect, since about half of those false alarms come out with the wrong sign by chance alone.

6. Independent trials and the complement rule (~2 min)

If an individual is detected with probability p on any one occasion, independently across k occasions, the probability of **at least one** detection is

$$1 - (1 - p)^k$$

(the complement of “missed on every single occasion,” and each miss is independent so the miss-probabilities multiply).

```

p_visit <- 0.15
k <- 1:6
round(1 - (1 - p_visit)^k, 3)

```

```
## [1] 0.150 0.278 0.386 0.478 0.556 0.623
```

Even a modest 15%-per-visit detection rate accumulates fast: six independent visits already give better than even odds of at least one detection. This is the exact logic behind any “probability of detecting a present individual across repeated survey occasions” calculation.

7. Writing and testing small functions, one at a time (~2 min, coding practice, not math)

Not a math topic, but worth a deliberate mention before Act 2’s Part 2, which is scaffolded into several small functions chained together: write **one** function that does one clearly-defined piece of the job, and test it immediately against an input where you already know the right answer by hand – *before* moving on to the next piece.

```
# toy example, unrelated to Act 2's actual functions: euclidean distance
dist2 <- function(x1, y1, x2, y2) sqrt((x2 - x1)^2 + (y2 - y1)^2)
dist2(0, 0, 3, 4)    # known answer: a 3-4-5 triangle, should return exactly 5

## [1] 5
```

Why this matters here specifically: four small functions get chained into one final power estimate in Act 2. A bug in any single piece doesn't usually error – it just quietly produces a wrong final number, and that's very hard to trace back once four functions are already composed. Checking each piece in isolation, on a case you can verify by hand, is what makes the bug traceable instead of invisible.

Take-home message

- Lognormal noise is multiplicative and has constant CV – the reason count indices get analyzed on the log scale.
- A confidence interval's "90%" describes the long-run behavior of the *procedure*, confirmed by simulating from a known truth.
- Power analysis simulates a plausible true effect under a candidate design and checks how often the planned analysis would catch it – it's a property of a design, not of data in hand.
- A directional decision rule (significant *and* correct sign) matters whenever the scientific question itself has a direction; skipping the sign check inflates false alarms.
- The complement rule $(1 - (1 - p)^k)$ turns single-occasion detection probabilities into multi-occasion ones.
- Build and test one function at a time – especially important once several get chained together, since a broken link rarely announces itself with an error.

Roadmap tie-in (which Act needs what)

- **Act 1:** lognormal noise model, log-linearization, CI coverage.
- **Act 2:** power analysis in general, the directional decision rule, the complement rule (its `detect_prob()` function *is* this), and the function-writing/testing workflow (its whole Part 2 structure).
- **Act 3 (optional):** the lognormal/variance-function material from today carries over directly; nothing new needed beyond today's review.

PART 2 – CHEAT SHEET (opening pass / circulating)

Copy for the podium or for circulating during lab. Full derivations, code, and rationale: see Part 1 above.

0. Recap (~1 min)

- Monday: simulating fake data can validate a fitting method, build intuition, forecast, or **power a design** – today builds the pieces Act 2 needs to do that last one

1. Lognormal review (~2 min)

$$\text{median} = e^m \quad \text{mean} = e^{m+s^2/2} \quad \text{SD} = \sqrt{e^{s^2} - 1} \times \text{mean}$$

- mean sits above median (right skew); **SD is a constant proportion of the mean**, not a fixed number of animals

2. Linearize the exponential trend (~3 min)

$$I_t = I_0 \lambda^t e^{\varepsilon_t} \Rightarrow \log(I_t) = \log(I_0) + bt + \varepsilon_t, \quad b = \log \lambda$$

- fit $\text{lm}(\log(I) \sim t)$, not $\text{lm}(I \sim t)$
- track which scale a number is on: $\hat{b}$ vs. $\hat{\lambda} = e^{\hat{b}}$ vs. percent change $1 - e^{T\hat{b}}$

3. Confidence interval coverage (~3 min)

- a 90% CI: **the procedure** catches the true value ~90% of the time across repeated studies – not “90% chance it’s in this one interval”
- checked by simulating from a *known* truth and tallying containment
- MC SE of a proportion: $\sqrt{p(1-p)/R}$ sets how many reps you need

4. Power analysis, in general (~4 min)

1. assume a **real** effect (not “no effect”)
2. simulate one dataset under a **candidate design**
3. apply the **real analysis method**; record hit/miss
4. repeat many times $\rightarrow$ power = fraction detected
5. compare designs $\rightarrow$ often a **cost/power trade-off search**

- example: ~16 years of annual monitoring needed for 80% power against a 40%-over-20-year decline (sage-grouse lek counts, review numbers)
- power is a property of a **design**, never of data you already have

5. Type I/II error & directional rules (~2 min)

- Type I = false alarm; Type II = missed detection; power = 1 – Type II
- “detected a **decline**” needs **two** things: significant, *and* right sign
- skipping the sign check roughly **doubles** the false-alarm rate under a true null (half of false alarms come out the wrong sign by chance)

6. Complement rule (~2 min)

$$P(\text{at least one detection in } k \text{ tries}) = 1 - (1 - p)^k$$

- $p = 0.15/\text{visit}$: $k = 1..6 \rightarrow 0.15, 0.28, 0.39, 0.48, 0.56, 0.62$
- “not missed on *every single* independent occasion”

7. Build and test one function at a time (~2 min, not math)

- write **one** piece $\rightarrow$ test on a case you can verify by hand $\rightarrow$ *then* move to the next piece
- Act 2 chains four functions together – an untested bug in one is invisible once composed, not an error, just a wrong final number

Act-by-Act quick reference

- **Act 1** – lognormal, log-linearization, CI coverage
- **Act 2** – power analysis, directional decision rule, complement rule (`detect_prob()`), test-as-you-build workflow
- **Act 3 (optional)** – today’s lognormal/variance material carries straight over, nothing further needed